The leak-free task was DATA-BOUND.

3x the data takes N=24 from 0.854 to 1.000.

On the SAME 1500-instance shared pool that scored the leak-free baseline at 0.8540: n_train 80000 -> 240000 gives 1.0000, delta +0.1460, 219 discordant instances won and 0 lost, McNemar exact p = 2.4e-66. Every instance in the pool, solved.

This is the prerequisite the project has been blocked on. After finding 16 closed the adjacency-order leak, the clean N=24 cell sat at 0.854 -- BELOW the 0.90 gate that defines L* -- so no layer threshold could be measured on the real task at all. It now clears the gate by 10 points at one change: more distinct instances.

And the cheap cell got there too. Cell B trains on the same 240000 instances for 100 epochs -- 24M samples, exactly the step count of the 0.854 run -- and also scores 1.0000. So the binding constraint was COVERAGE, not gradient steps, and the baseline's 300 epochs over 80000 instances were two thirds wasted compute.

job 49705064, COMPLETED 23:35:04, ExitCode 0:0 | commit 9781a31 | scripts/slurm/sudoku2_n24_leakfree_data.sbatch | seed 0 | shuffle_adj=True

What the model actually sees

Every token is one categorical id into a 34-symbol vocabulary (N=24, q=4, degree=5). No text, no digits-as-characters.

VOCABULARY

0-5	BEGIN_HINTS BEGIN_GRAPH BEGIN_REASONING BEGIN_ROUND SOLVED UNRESOLVED	*section + round markers*
6-9	COLOR_0 COLOR_1 COLOR_2 COLOR_3	*the q colours*
10-33	N0 N1 ... N23	*one id per node, no structure in the id*

GRAMMAR

BEGIN_HINTS	(NODE COLOR)*
BEGIN_GRAPH	(NODE (NODE){degree})*
BEGIN_REASONING	(BEGIN_ROUND (NODE COLOR)+)*
SOLVED	

The graph section is a neighbour table, not an image: node, then its degree neighbours. THE ORDER WITHIN A ROW IS THE LEAK of finding 16.

THIS INSTANCE, 204 TOKENS (seed 50000000038)

HINTS	BEGIN_HINTS N0 COLOR_2 N1 COLOR_1 N4 COLOR_1 N5 COLOR_1 N7 COLOR_2 N9 COLOR_0 ... (23 tokens)
GRAPH	BEGIN_GRAPH N0 N17 N21 N6 N13 N9 N1 N19 N12 N6 N7 N14 ... (145 tokens)
REASONING	BEGIN_REASONING BEGIN_ROUND N3 COLOR_3 N21 COLOR_3 N23 COLOR_3 BEGIN_ROUND N6 COLOR_0 N15 COLOR_1 ... (36 tokens)

Loss is on the REASONING section only: 8 rounds, 13 deductions, 26 target tokens. Everything before BEGIN_REASONING is the problem statement.

One real reasoning trace

11 hints of 24 nodes, then 8 rounds of simultaneous deduction. Hints: N0=2 N1=1 N4=1 N5=1 N7=2 N9=0 N10=2 N12=0 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND          N3 COLOR_3 N21 COLOR_3 N23 COLOR_3
  3 deduced
BEGIN_ROUND          N6 COLOR_0 N15 COLOR_1
  2 deduced
BEGIN_ROUND          N11 COLOR_3 N13 COLOR_0
  2 deduced
BEGIN_ROUND          N18 COLOR_2 N20 COLOR_0
  2 deduced
... 4 further rounds

SOLVED
```

WHY ONE DEDUCTION FIRES

node N3, neighbours [17, 4, 10, 19, 5]

colours known before this round:

```
   [0, 1, 2]  (3 = q-1)
```

only colour left: 3

```
so the trace emits  N3 COLOR_3
```

The model is never shown this reasoning -- only the node/colour pair.

A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

T=8 rounds, the 90th percentile of 40 draws from this config (range 1-10); seed 50000000038, shuffle_adj=True; evidence from sudoku_tokens.evidence_rounds (pre-round state, as DD saw it)

Metrics of record

One pool, 1500 instances at seed_start=5e10, scored by all three checkpoints; paired McNemar exact.

cell	solve_rate	95% Wilson	delta	won/lost	McNemar p	val_nll	val_acc	mem gap
80k x 300ep (finding 17)	0.8540	[0.8352, 0.8710]	reference	--	--	1.3087	0.5086	+0.1000
240k x 300ep (cell A)	1.0000	[0.9974, 1.0000]	+0.1460	219/0	2.4e-66	1.2498	0.5170	+0.0000
240k x 100ep (cell B)	1.0000	[0.9974, 1.0000]	+0.1460	219/0	2.4e-66	1.2503	0.5194	+0.0000

THE COMPARISON IS AS TIGHT AS THIS PROJECT CAN MAKE IT

- Same instance distribution (N=24, degree 5, q=4, rho 0.30-0.95, removal hints, shuffle_adj=True), so sudoku_agg_eval's check_comparable allows the pool at all.
- Same architecture, optimizer and seed: d_model 256, 4 heads, L=12, ff=1024, pre-norm, lr=1e-3 constant, batch 32, seed 0. n_train and epochs are the only knobs that moved.
- Cells A and B solve the SAME 1500 instances, so they do not merely tie on the rate -- they agree instance by instance (0 discordant pairs between them).

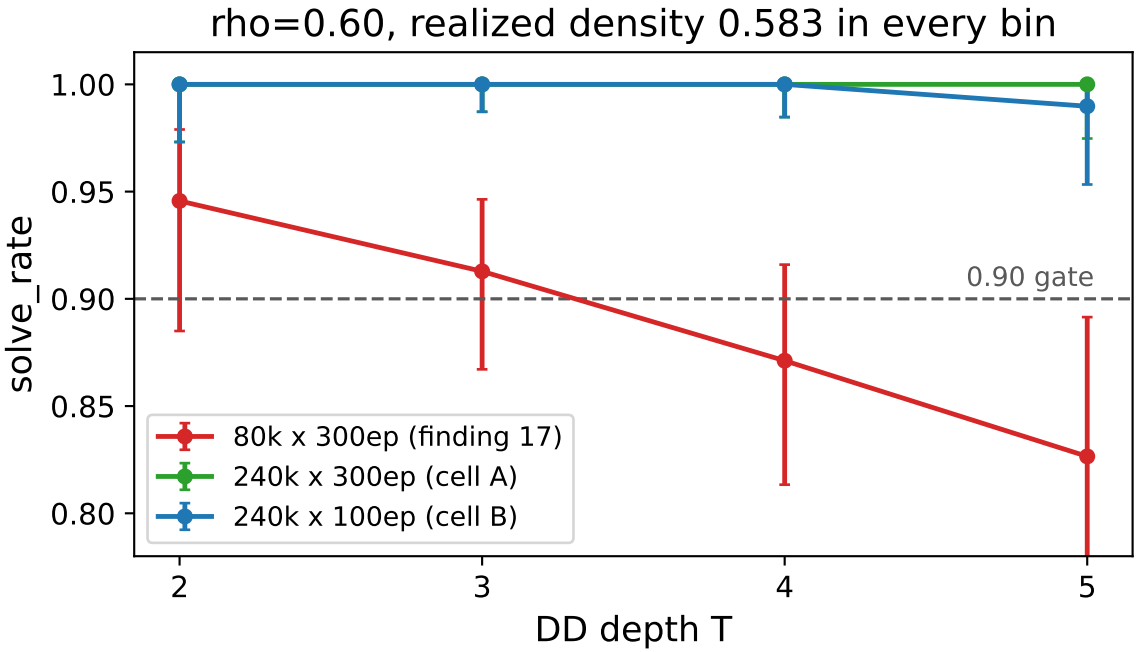
ONE HONEST ASTERISK ON 'IDENTICAL ARCHITECTURE'

max_len is computed over the TRAINING pool, and a 240000-instance pool contains a slightly longer trace than an 80000-instance one: 210 -> 212 tokens, so n_params goes 9,548,834 -> 9,549,346 (+512 = two positional embedding rows, 0.005%). It also gives the data cells a 2-token longer generation budget. Neither can account for +0.146; truncated_rate is 0.0000 for all three cells, so nothing was ever cut off at either budget.

python -m spin_gnn.sudoku_agg_eval --n-instances 1500 --seed-start 50000000000; mem gap from sudoku_memorization_eval --n-memorization 600

The depth cost was a data artefact

Pre-registration (d) was the control I most wanted to survive. It did not.



THE SAME THING HAPPENS AT rho=0.75.

The baseline fell 1.000 / 0.969 / 0.890 over T=1,2,3; both data cells score 1.000 in all three bins. Cell B's only miss anywhere is that one instance at rho=0.60, T=5, so even the cheap cell is within one instance of a clean sweep of all 1096 depth-eval instances.

	T=2	T=3	T=4	T=5	Pearson r
80k x 300ep	0.9457	0.9128	0.8712	0.8265	-0.998
240k x 300ep	1.0000	1.0000	1.0000	1.0000	n/a (flat)
240k x 100ep	1.0000	1.0000	1.0000	0.9898	-0.775

n per bin: T=2:92, T=3:195, T=4:163, T=5:98 (548 per row)

WHAT I PREDICTED, AND WHY IT WAS WRONG

Prediction: more data lifts every bin by about the same amount and the slope persists, because finding 13 showed the depth cost is serial computation and no amount of data fixes a model that is too shallow. Falsifier: Pearson r at rho=0.60 above -0.5.

What happened: the deep bins did not just improve, they SATURATED. L=12 was never too shallow for T<=5; it had not seen enough instances to cover the deep ones. Note the pre-registered STATISTIC is uninformative here: cell B's r = -0.775 is three bins at exactly 1.000 plus ONE missed instance in 548, so it does not clear -0.5 even though the cost it was built to detect is gone.

Consequence: the leak-free L*(T) ladder at N=24 now has NOTHING TO MEASURE -- every cell would sit at 1.000. It has to move to N=48 or N=96, which finding 22 already argued.

sudoku_depth_eval --rho 0.75 0.60 --n-pool 600 --min-per-bin 40, same generator stream as the baseline

Pre-registrations, as resolved

Written into scripts/slurm/sudoku2_n24_leakfree_data.sbatch before the job ran.

	claim	verdict	evidence
(a)	Cell A clears 0.90 on the shared pool	CONFIRMED	1.0000 vs 0.8540, +0.1460, p=2.4e-66
(b)	Cell B also beats 0.8540, BY LESS than A	FALSIFIED (the flagged half)	B beats it by the same +0.1460; A and B agree on all 1500 instances. The falsifier I named -- 'B >= A, the extra epochs do nothing' -- is what happened.
(c)	The memorization gap shrinks below +0.1000	CONFIRMED	+0.0000 (train 1.0000 vs held-out 1.0000, p=1.0, n=600), both cells
(d)	The depth cost survives: r at rho=0.60 stays below -0.5	FALSIFIED (in substance)	All four bins at 1.0000 in cell A, so the cost is gone. The pre-registered r test cannot say so: cell B's r=-0.775 is one missed instance in 548 and still 'passes'.

MY RECORD ON DATA-BUDGET PREDICTIONS IS NOW 3 FOR 8, ALL ONE DIRECTION.

The sbatch header said it before the job: 'I have twice underestimated how much data this task wants.' Three times, now -- (b) and (d) are the same mistake again, and (d) is the expensive one, because finding 17's depth claim was the first clean evidence in this project that reasoning depth is what costs, and at N=24 it is now attributable to coverage instead. The claim is not dead, just unmeasured at a budget that can hold it.

pool never trained or tuned on: seed_start 5e10 vs training seeds 0-239999

What this changes, and what it does not

WHAT IT CHANGES

- The leak-free N=24 task is SOLVED, not merely improved: 1500/1500 on a held-out pool, with the adjacency-order shortcut of finding 16 removed from training.
- The working budget for every future leak-free run is 240000 instances x 100 epochs. That is the number scripts/slurm/sudoku2_leakfree_lstar.sbatch has been waiting for.
- Finding 17's 0.854 is demoted from 'the cost of removing the leak' to 'the cost of removing the leak AT 80000 instances'. The leak was worth data, not capability.
- Finding 17's depth cost at N=24 is withdrawn (see the previous page).

WHAT IT DOES NOT

- It says nothing about N=48 or N=96, where the traces are 2-4x longer and 240000 instances is 3-12x the GPU time. The budget transfers as a hypothesis, not a fact.
- A ceiling result cannot be ranked. Anything at 1.000 on 1500 instances ties anything else at 1.000, so this pool can no longer separate N=24 cells from each other and the metric of record has to move to a harder distribution (lower rho, or larger N).
- One seed, as always in this project.

WHAT I WOULD DO NEXT

1. Move the leak-free L*(T) ladder to N=48 at 240000 x 100ep -- the budget is now known and the T range there (roughly 3-7) is wide enough to split, unlike N=24's 2-5.
2. Re-measure the N=24 leak-free depth cost at a density where it is not at ceiling (rho=0.45), which is one depth_eval call against the cell B checkpoint.
3. Rerun the SAE and probe analyses on cell B: the interpretability results so far were all read off a model that only solved 85% of the task.

cell B is the checkpoint to keep: same 1.0000 at a third of the compute (d0256_h04_L12_ff01024_lr1e-03_pre_shuf in data240k_ep100)